

Lesson 14 Structured Note: Associativity and Unary Operators

0. Current Position in the Course

You are now in **Lesson 14** of the stack curriculum.

Before Lesson 14, you already learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 7	A complete C linked-list stack uses <code>struct Node</code> , a global <code>top</code> pointer, <code>push</code> , <code>pop</code> , <code>display</code> , and <code>main</code> .
Lesson 8	A C++ class can be used to implement a linked-list stack more safely.
Lesson 9	Parenthesis matching was introduced as a stack application.
Lesson 10	Parenthesis matching was implemented using an array character stack.
Lesson 11	Parenthesis matching was implemented using a linked-list character stack.
Lesson 12	Bracket matching was extended to <code>()</code> , <code>[]</code> , and <code>{}</code> .
Lesson 13	You learned infix, prefix, postfix, precedence, and simple manual conversion.

Lesson 14 continues:

Phase 4: Expression Notation and Conversion

In Lesson 13, you learned that expressions can be written in three forms:

Notation	Operator position	Example
Infix	Operator is between operands	$A + B$
Prefix	Operator is before operands	$+ A B$
Postfix	Operator is after operands	$A B +$

Lesson 13 also introduced **precedence**, which means operator priority.

For example:

$A + B * C$

Because $*$ has higher precedence than $+$, the expression means:

$A + (B * C)$

Lesson 14 adds two important ideas:

1. **Associativity**
2. **Unary operators**

These ideas are needed before learning infix-to-postfix conversion using a stack.

1. Lesson 14 Goal

By the end of this lesson, you should understand:

- What associativity means.
- The difference between precedence and associativity.
- Why associativity matters when operators have the same precedence.
- What left-to-right associativity means.
- What right-to-left associativity means.
- How to group expressions such as $A - B - C$.
- How to group expressions such as $A = B = C$.
- How to group expressions such as $A \wedge B \wedge C$.
- What unary operators are.
- The difference between unary operators and binary operators.
- The difference between unary minus and binary minus.
- How unary operators affect prefix and postfix notation.

- How to convert expressions with associativity into postfix form.
- How to dry-run a complex expression such as `-A + B * log N!`.
- Why Lesson 14 is important before coding infix-to-postfix conversion.

2. Quick Review: What Is an Expression?

An **expression** is a combination of operands and operators.

Example:

`A + B`

This expression means:

Add A and B

An expression usually has two main parts:

Part	Simple meaning	Examples
Operand	A value or variable used in an expression	<code>A</code> , <code>B</code> , <code>C</code> , <code>5</code> , <code>x</code>
Operator	A symbol or word that performs an operation	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>^</code> , <code>=</code> , <code>log</code>

In this expression:

`A + B`

- `A` is an operand.
- `+` is an operator.
- `B` is an operand.

3. Quick Review: Infix, Prefix, and Postfix

3.1 Infix Notation

In **infix** notation, the operator is written between the operands.

Structure:

Operand Operator Operand

Example:

A + B

The operator **+** is between **A** and **B**.

So this is infix notation.

3.2 Prefix Notation

In **prefix** notation, the operator is written before the operands.

Structure:

Operator Operand Operand

Example:

+ A B

This means the same thing as:

A + B

3.3 Postfix Notation

In **postfix** notation, the operator is written after the operands.

Structure:

Operand Operand Operator

Example:

A B +

This also means the same thing as:

A + B

3.4 Same Expression in Three Forms

Infix	Prefix	Postfix
A + B	+ A B	A B +
A - B	- A B	A B -
A * B	* A B	A B *
A / B	/ A B	A B /

Important idea:

Infix, prefix, and postfix are not different mathematics. They are different ways of writing the same expression.

4. Precedence

4.1 Meaning of Precedence

Precedence means operator priority.

It answers this question:

Which operator should be handled first?

Example:

A + B * C

There are two operators:

+
*

Multiplication $*$ has higher precedence than addition $+$.

So this expression is grouped as:

$A + (B * C)$

not:

$(A + B) * C$

4.2 Basic Precedence Table

A beginner-friendly precedence table is:

Priority	Operators	Meaning
Highest	$()$	Brackets are handled first.
High	Unary operators such as $-A$, $!A$, $\log X$, $N!$	Operators working on one operand.
High	$^$	Power/exponent operator.
Medium	$*$, $/$	Multiplication and division.
Low	$+$, $-$	Addition and subtraction.
Very low	$=$	Assignment.

This table helps us decide which operation happens first.

Example:

$A + B * C$

Since $*$ has higher precedence than $+$, we do:

$B * C$

first.

5. Associativity

5.1 Meaning of Associativity

Associativity means the direction used when operators have the same precedence.

It answers this question:

If two operators have the same priority, which one should we handle first?

Example:

A - B - C

Both operators are `-`.

So they have the same precedence.

Precedence alone cannot decide the grouping.

We need associativity.

5.2 Precedence vs Associativity

Precedence and associativity are different.

Idea	Question it answers	Example
Precedence	Which operator has higher priority?	In <code>A + B * C</code> , <code>*</code> is handled before <code>+</code> .
Associativity	If operators have the same priority, which direction do we use?	In <code>A - B - C</code> , do we group from left or right?

Simple rule:

Precedence chooses the stronger operator. Associativity chooses the direction when there is a tie.

6. Left-to-Right Associativity

6.1 Meaning

Left-to-right associativity means:

When operators have the same precedence, group from the left side first.

Example:

$A - B - C$

The two $-$ operators have the same precedence.

Because subtraction is left-to-right, we group it as:

$(A - B) - C$

not:

$A - (B - C)$

6.2 Operators That Usually Use Left-to-Right Associativity

Common left-to-right operators include:

$+$
 $-$
 $*$
 $/$

Examples:

Expression	Correct grouping
$A + B - C$	$(A + B) - C$

Expression	Correct grouping
$A - B + C$	$(A - B) + C$
$A * B / C$	$(A * B) / C$
$A / B * C$	$(A / B) * C$

7. Left-to-Right Example 1: $A + B - C$

Expression:

$A + B - C$

The operators are:

$+$
 $-$

Both $+$ and $-$ have the same precedence.

They are handled from left to right.

So we group the expression as:

$(A + B) - C$

7.1 Convert $A + B - C$ to Postfix

First handle:

$A + B$

Postfix:

$A B +$

Now subtract C :

$$(A + B) - C$$

Postfix:

$$A B + C -$$

Final answer:

$$A + B - C \rightarrow A B + C -$$

8. Left-to-Right Example 2: $A - B - C$

Expression:

$$A - B - C$$

Both operators are $-$.

Subtraction is left-to-right.

So we group it as:

$$(A - B) - C$$

8.1 Convert $A - B - C$ to Postfix

First handle:

$$A - B$$

Postfix:

A B -

Now subtract C:

(A - B) - C

Postfix:

A B - C -

Final answer:

A - B - C -> A B - C -

9. Left-to-Right Example 3: A * B / C

Expression:

A * B / C

The operators are:

*

/

Both * and / have the same precedence.

Multiplication and division are handled from left to right.

So we group the expression as:

(A * B) / C

9.1 Convert $A * B / C$ to Postfix

First handle:

$A * B$

Postfix:

$A B *$

Now divide by C :

$(A * B) / C$

Postfix:

$A B * C /$

Final answer:

$A * B / C \rightarrow A B * C /$

10. Right-to-Left Associativity

10.1 Meaning

Right-to-left associativity means:

When operators have the same precedence, group from the right side first.

Example:

$A = B = C$

Assignment is right-to-left.

So we group it as:

$$A = (B = C)$$

not:

$$(A = B) = C$$

10.2 Operators That Often Use Right-to-Left Associativity

Important right-to-left operators in this lesson include:

$$\begin{array}{l} = \\ ^ \end{array}$$

Meaning:

Operator	Name	Associativity
=	Assignment	Right-to-left
^	Power/exponent	Right-to-left in this lesson

11. Right-to-Left Example 1: Assignment

Expression:

$$A = B = C = 5$$

Assignment $=$ is handled right to left.

So the expression is grouped as:

$$A = (B = (C = 5))$$

11.1 Meaning of $A = B = C = 5$

This means:

1. Assign 5 to C.
2. Then assign the result to B.
3. Then assign the result to A.

So the direction is from the right side toward the left side.

11.2 Convert $A = B = C = 5$ to Postfix

Start with the innermost part:

C = 5

Postfix:

C 5 =

Now handle:

B = (C = 5)

Postfix:

B C 5 = =

Now handle:

A = (B = (C = 5))

Postfix:

A B C 5 = = =

Final answer:

$A = B = C = 5 \rightarrow A B C 5 = = =$

12. Right-to-Left Example 2: Power Operator

Expression:

$A \wedge B \wedge C$

The operator $\wedge$ means power.

In this lesson, the power operator is right-to-left.

So the expression is grouped as:

$A \wedge (B \wedge C)$

not:

$(A \wedge B) \wedge C$

12.1 Why This Matters

The two groupings are different.

$A \wedge (B \wedge C)$

means:

Raise B to the power C first.
Then raise A to that result.

But:

$(A \wedge B) \wedge C$

means:

Raise A to the power B first.
Then raise that result to the power C.

These are not always the same.

So associativity matters.

12.2 Convert $A \wedge B \wedge C$ to Postfix

Because $\wedge$ is right-to-left, start with:

$B \wedge C$

Postfix:

$B C \wedge$

Now combine with $A \wedge$:

$A \wedge (B \wedge C)$

Postfix:

$A B C \wedge \wedge$

Final answer:

$A \wedge B \wedge C \rightarrow A B C \wedge \wedge$

13. Important Associativity Summary

Operator group	Example	Associativity	Correct grouping
Addition/subtraction	$A + B - C$	Left-to-right	$(A + B) - C$

Operator group	Example	Associativity	Correct grouping
Subtraction/subtraction	$A - B - C$	Left-to-right	$(A - B) - C$
Multiplication/division	$A * B / C$	Left-to-right	$(A * B) / C$
Assignment	$A = B = C$	Right-to-left	$A = (B = C)$
Power	$A ^ B ^ C$	Right-to-left	$A ^ (B ^ C)$

14. What Is a Unary Operator?

14.1 Meaning

A **unary operator** is an operator that works on **one operand**.

The word **unary** means:

one

So a unary operator needs only one value.

Example:

-A

This means:

negative A

The operator $-$ works only on A .

So this $-$ is unary.

14.2 Binary Operator

A **binary operator** works on **two operands**.

The word **binary** means:

two

Example:

A + B

The operator $+$ works on two operands:

A and B

So $+$ is binary here.

14.3 Unary vs Binary Operators

Type	Number of operands	Example	Meaning
Unary operator	1	$-A$	negative A
Binary operator	2	$A - B$	A minus B

Important idea:

The same symbol can sometimes be unary and sometimes binary.

The best example is $-$.

15. Unary Minus vs Binary Minus

This is one of the most important parts of Lesson 14.

The minus symbol $-$ can have two meanings.

15.1 Binary Minus

Example:

A - B

This means:

A minus B

The operator has two operands:

A and B

So this is binary minus.

Postfix form:

A B -

Prefix form:

- A B

15.2 Unary Minus

Example:

-A

This means:

negative A

The operator has only one operand:

A

So this is unary minus.

Postfix form can be written as:

A -

But this may confuse beginners because `-` also means binary subtraction.

So it is clearer to write unary minus as:

NEG

Then:

-A -> A NEG

Prefix form:

NEG A

15.3 How to Recognize Unary Minus

A minus sign is usually unary when it appears before a single operand and does not have a left operand.

Examples:

-A

Here, `-` is unary because it comes before `A`.

A * -B

Here, the second `-` is unary because it means negative `B`.

(-A)

Here, `-` is unary because it means negative `A`.

A minus sign is binary when it appears between two operands.

Example:

A - B

Here, **-** is binary because it subtracts **B** from **A**.

16. Common Unary Operators

Lesson 14 discusses unary operators such as:

-A
!A
log X
N!
*p

These operators do not all appear in the same position.

Some appear before the operand.

Some appear after the operand.

16.1 Prefix Unary Operators

A prefix unary operator comes before its operand.

Examples:

-A
!A
log X
*p

Meanings:

Expression	Meaning
-A	negative A
!A	not A

Expression	Meaning
<code>log X</code>	logarithm of X
<code>*P</code>	value pointed to by P, in pointer-style notation

16.2 Postfix Unary Operators

A postfix unary operator comes after its operand.

Example:

`N!`

This means:

factorial of N

The operator `!` comes after `N`.

So it is a postfix unary operator.

17. Unary Operator Prefix and Postfix Forms

Infix-style expression	Meaning	Prefix form	Postfix form
<code>-A</code>	negative A	<code>NEG A</code>	<code>A NEG</code>
<code>!A</code>	not A	<code>! A</code>	<code>A !</code>
<code>log X</code>	logarithm of X	<code>log X</code>	<code>X log</code>
<code>N!</code>	factorial of N	<code>! N</code>	<code>N !</code>
<code>*P</code>	dereference P	<code>DEREF P</code>	<code>P DEREF</code>

Important note:

For beginner clarity, it is better to write unary minus as `NEG` instead of just `-`.

Why?

Because this:

A B -

usually means:

A - B

But this:

A NEG

clearly means:

-A

18. Unary Operators and Precedence

Unary operators usually have high precedence.

That means they are usually handled before lower-precedence binary operators like `+`, `-`, `*`, and `/`.

Example:

-A + B

The unary minus applies to `A` first.

So the expression is grouped as:

(-A) + B

not:

-(A + B)

18.1 Convert $-A + B$ to Postfix

First handle:

$-A$

Postfix:

$A \text{ NEG}$

Now add B :

$(-A) + B$

Postfix:

$A \text{ NEG } B +$

Final answer:

$-A + B \rightarrow A \text{ NEG } B +$

19. Factorial as a Unary Operator

Factorial is written after the operand.

Example:

$N!$

This means:

factorial of N

It has one operand:

N

So factorial is a unary operator.

Postfix form:

N !

Prefix form:

! N

20. Log as a Unary Operator

The expression:

log N

means:

logarithm of N

The operator `log` works on one operand:

N

So `log` is unary.

Postfix form:

N log

Prefix form:

log N

21. Complex Example: $-A + B * \log N!$

Now we combine many Lesson 14 ideas.

Expression:

$-A + B * \log N!$

This expression includes:

Part	Type
$-A$	Unary minus
$*$	Binary multiplication
$\log$	Unary operator
$N!$	Unary factorial
$+$	Binary addition

21.1 Decide the Order of Operations

Expression:

$-A + B * \log N!$

The order is:

1. $N!$
2. $\log N!$
3. $-A$
4. $B * \log N!$
5. $-A + B * \log N!$

Why?

Because:

- Factorial applies directly to **N**.
 - **log** applies to the result of **N!**.
 - Unary minus applies to **A**.
 - Multiplication has higher precedence than addition.
 - Addition happens last.
-

21.2 Add Grouping Marks

Original expression:

$-A + B * \log N!$

Grouped expression:

$(-A) + (B * (\log (N!)))$

This grouping shows the real order clearly.

21.3 Convert Step by Step to Postfix

Step 1: Convert **-A**

Infix:

$-A$

Postfix:

$A \text{ NEG}$

Step 2: Convert **N!**

Infix:

N!

Postfix:

N !

Step 3: Convert `log N!`

We already converted:

N!

as:

N !

Now apply `log` to it.

Postfix:

N ! log

Step 4: Convert `B * log N!`

Left part:

B

Right part:

N ! log

The operator is:

*

Postfix:

B N ! log *

Step 5: Add the two large parts

Left part:

A NEG

Right part:

B N ! log *

The final operator is:

+

Final postfix:

A NEG B N ! log * +

So:

-A + B * log N! -> A NEG B N ! log * +

22. Same Complex Example in Prefix Form

Expression:

-A + B * log N!

Grouped form:

$(-A) + (B * (\log (N!)))$

Now convert to prefix.

22.1 Convert -A

Prefix:

NEG A

22.2 Convert N!

Prefix:

! N

22.3 Convert log N!

Prefix:

log ! N

22.4 Convert B * log N!

Prefix:

* B log ! N

22.5 Convert the full expression

The main operator is `+`.

Left part:

NEG A

Right part:

* B log ! N

Final prefix:

+ NEG A * B log ! N

So:

-A + B * log N! -> + NEG A * B log ! N

23. Why Associativity Matters for Postfix Conversion

When converting infix to postfix, the program must decide when to move operators from the stack to the output.

This is easy when one operator has higher precedence than another.

Example:

A + B * C

Since `*` has higher precedence than `+`, multiplication must happen before addition.

But associativity becomes important when operators have equal precedence.

Example:

A - B - C

Both operators are $-$.

Since $-$ is left-to-right, the first $-$ should be handled before the second $-$.

So the postfix is:

A B - C -

But for this expression:

A ^ B ^ C

Both operators are $^$.

Since $^$ is right-to-left, the second $^$ is handled first.

So the postfix is:

A B C ^ ^

Important idea:

In stack-based infix-to-postfix conversion, equal precedence is not enough. Associativity tells us what to do when precedence is equal.

24. Why Unary Operators Matter for Postfix Conversion

A simple infix-to-postfix algorithm often assumes that every operator is binary.

That means it expects every operator to use two operands.

For example:

A + B

The operator $+$ uses:

A and B

But unary operators use only one operand.

Example:

$-A$

The unary minus uses only:

A

This can confuse a binary-only conversion algorithm.

So unary operators need special handling.

24.1 Example of the Problem

Expression:

$A - B$

Here $-$ is binary.

Postfix:

$A B -$

Expression:

$-A$

Here $-$ is unary.

Postfix:

A NEG

The same symbol $-$ has two different meanings.

So a program must know which meaning is being used.

25. How to Think Like a Beginner During Conversion

When converting manually, use this method:

1. Look for brackets first.
2. Look for unary operators.
3. Look for higher-precedence operators.
4. If operators have the same precedence, use associativity.
5. Convert the smallest part first.
6. Treat that converted part as one unit.
7. Continue until the full expression is converted.

Example:

$A + B * C$

Step 1:

$B * C$

happens first.

Postfix:

$B C *$

Step 2:

$A + (B * C)$

Postfix:

A B C * +

26. Required Dry Runs

26.1 Dry Run: A + B - C

Expression:

A + B - C

Operators:

+
-

Both have the same precedence.

They are left-to-right.

Grouping:

(A + B) - C

Postfix steps:

Step	Part	Postfix
1	A + B	A B +
2	(A + B) - C	A B + C -

Final postfix:

A B + C -

26.2 Dry Run: A - B - C

Expression:

A - B - C

Both operators are -.

Subtraction is left-to-right.

Grouping:

(A - B) - C

Postfix steps:

Step	Part	Postfix
1	A - B	A B -
2	(A - B) - C	A B - C -

Final postfix:

A B - C -

26.3 Dry Run: A = B = C

Expression:

A = B = C

Both operators are =.

Assignment is right-to-left.

Grouping:

A = (B = C)

Postfix steps:

Step	Part	Postfix
1	B = C	B C =
2	A = (B = C)	A B C = =

Final postfix:

A B C = =

26.4 Dry Run: A ^ B ^ C

Expression:

A ^ B ^ C

Both operators are ^.

Power is right-to-left in this lesson.

Grouping:

A ^ (B ^ C)

Postfix steps:

Step	Part	Postfix
1	B ^ C	B C ^
2	A ^ (B ^ C)	A B C ^ ^

Final postfix:

A B C ^ ^

26.5 Dry Run: $-A + B$

Expression:

$-A + B$

Unary minus applies to A first.

Grouping:

$(-A) + B$

Postfix steps:

Step	Part	Postfix
1	$-A$	$A \text{ NEG}$
2	$(-A) + B$	$A \text{ NEG } B +$

Final postfix:

$A \text{ NEG } B +$

26.6 Dry Run: $-A + B * \log N!$

Expression:

$-A + B * \log N!$

Correct grouping:

$(-A) + (B * (\log (N!)))$

Order:

Step	Operation	Resulting postfix part
1	-A	A NEG
2	N!	N !
3	log N!	N ! log
4	B * log N!	B N ! log *
5	Full expression	A NEG B N ! log * +

Final postfix:

A NEG B N ! log * +

27. Common Beginner Mistakes

Mistake 1: Thinking Precedence and Associativity Are the Same

Wrong idea:

Precedence and associativity both mean priority.

Correct idea:

Precedence means priority.
Associativity means direction when priorities are equal.

Mistake 2: Grouping $A - B - C$ from the Right

Wrong grouping:

$A - (B - C)$

Correct grouping:

$$(A - B) - C$$

Why?

Because subtraction is left-to-right.

Mistake 3: Grouping $A \wedge B \wedge C$ from the Left

Wrong grouping:

$$(A \wedge B) \wedge C$$

Correct grouping:

$$A \wedge (B \wedge C)$$

Why?

Because power is right-to-left in this lesson.

Mistake 4: Treating Every Minus Sign as Binary Minus

Wrong idea:

Every - means subtraction.

Correct idea:

- can be unary or binary.

Examples:

Expression	Meaning	Type of $-$
$A - B$	A minus B	Binary
$-A$	negative A	Unary

Mistake 5: Forgetting That Unary Operators Have One Operand

Wrong idea:

Every operator needs two operands.

Correct idea:

Binary operators need two operands.
Unary operators need one operand.

Mistake 6: Writing Unary Minus as $-$ Without Clarifying It

This postfix expression can be confusing:

A $-$

It may look incomplete or confusing because $-$ usually means binary subtraction.

Clearer version:

A NEG

This clearly means:

$-A$

28. Lesson 14 Key Tables

28.1 Associativity Table

Operator	Name	Associativity
$+$	Addition	Left-to-right
$-$	Subtraction	Left-to-right

Operator	Name	Associativity
*	Multiplication	Left-to-right
/	Division	Left-to-right
^	Power	Right-to-left in this lesson
=	Assignment	Right-to-left

28.2 Unary and Binary Table

Expression	Operator	Number of operands	Type
A + B	+	2	Binary
A - B	-	2	Binary
-A	-	1	Unary
!A	!	1	Unary
N!	!	1	Unary
log N	log	1	Unary

28.3 Postfix Summary Table

Infix	Correct grouping	Postfix
A + B - C	(A + B) - C	A B + C -
A - B - C	(A - B) - C	A B - C -
A * B / C	(A * B) / C	A B * C /
A = B = C	A = (B = C)	A B C = =
A ^ B ^ C	A ^ (B ^ C)	A B C ^ ^
-A + B	(-A) + B	A NEG B +
-A + B * log N!	(-A) + (B * (log (N!)))	A NEG B N ! log * +

29. Why Lesson 14 Is Important Before Lesson 15

Lesson 15 will teach **infix-to-postfix conversion using a stack**.

That algorithm must decide what to do when it sees operators.

For example, if the current operator is `-` and the stack already has `-`, should the algorithm pop the old operator first?

For left-to-right operators, yes, same-precedence operators are usually popped first.

Example:

A - B - C

Postfix:

A B - C -

But for right-to-left operators like `^`, the same rule is different.

Example:

A ^ B ^ C

Postfix:

A B C ^ ^

So the stack conversion algorithm must understand:

1. Precedence
2. Associativity
3. Unary vs binary operators

Without Lesson 14, the conversion program may produce wrong postfix expressions.

30. Beginner Checkpoints

You understand Lesson 14 if you can answer these questions:

1. What is precedence?
 2. What is associativity?
 3. What is the difference between precedence and associativity?
 4. Why is $A - B - C$ grouped as $(A - B) - C$?
 5. Why is $A = B = C$ grouped as $A = (B = C)$?
 6. Why is $A \wedge B \wedge C$ grouped as $A \wedge (B \wedge C)$ in this lesson?
 7. What is a unary operator?
 8. What is a binary operator?
 9. What is the difference between $A - B$ and $-A$?
 10. Why is $A \text{ NEG}$ clearer than $A -$ for unary minus in postfix notation?
 11. What is the postfix form of $-A + B$?
 12. What is the postfix form of $-A + B * \log N!$?
-

31. Practice Tasks

Task 1

Convert this expression to postfix:

$A + B - C$

Expected answer:

$A B + C -$

Task 2

Convert this expression to postfix:

$A - B - C$

Expected answer:

A B - C -

Task 3

Convert this expression to postfix:

A = B = C

Expected answer:

A B C = =

Task 4

Convert this expression to postfix:

A ^ B ^ C

Expected answer:

A B C ^ ^

Task 5

Convert this expression to postfix:

-A + B

Expected answer:

A NEG B +

Task 6

Convert this expression to postfix:

$-A + B * \log N!$

Expected answer:

$A \text{ NEG } B \text{ N ! } \log * +$

32. Final Lesson Summary

Lesson 14 teaches that expression conversion needs more than just precedence.

You also need associativity.

Precedence tells you which operator has higher priority.

Example:

$A + B * C$

Because $*$ has higher precedence than $+$, the grouping is:

$A + (B * C)$

Associativity tells you what direction to use when operators have the same precedence.

Example:

$A - B - C$

Because $-$ is left-to-right, the grouping is:

$(A - B) - C$

But:

$A \wedge B \wedge C$

is grouped as:

$A \wedge (B \wedge C)$

because $\wedge$ is right-to-left in this lesson.

Lesson 14 also teaches **unary operators**.

A unary operator works on one operand.

Example:

$-A$

means:

negative A

A binary operator works on two operands.

Example:

$A - B$

means:

A minus B

The main takeaway is:

Precedence chooses which operator type goes first. Associativity chooses the direction when operators have the same precedence. Unary operators use one operand, so they must be handled differently from binary operators.